

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Text Search Indexer"

SUBMITTED BY:

Student Name: Dharaniselvi M
Roll Number: 2303820820522031
Program: B.Tech
Branch: B.Tech(Information Technology)

SUBMITTED TO:

E.G.S. Pillay Engineering College

Internship Details:

AICTE Student ID: STU68fe3c1c666041761491996
APAAR ID: 570038967555
Internship Duration: 8 Weeks (Jun 2026 - Jul 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Dharaniselvi M**, Roll No. **2303820820522031**, a student of **E.G.S. Pillay Engineering College**, has successfully completed **8-Week (Jun 2026 - Jul 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**Text Search Indexer**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera

Mentor, EduSkills Academy

Date: 03-08-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Dharaniselvi M

(Signature)

Date: 03-08-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Information Technology)** for their encouragement and support.

ABSTRACT

This project details the development of a Text Search Indexer designed to process and query multiple text inputs efficiently. The system tokenizes and normalizes input text, constructs a word-to-line index for rapid retrieval, and supports both single and multi-keyword searches. The outcome is a robust search mechanism capable of returning relevant results along with summaries of keyword occurrence counts, demonstrating effective text processing and information retrieval techniques.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **MERN full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **8 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The Text Search Indexer is a sophisticated system engineered to process and analyze large volumes of textual data. Its primary function is to create an efficient index that allows for rapid searching across multiple text documents. The system meticulously breaks down text into individual words (tokenization) and standardizes them (normalization) to ensure search accuracy. It then builds an inverted index where each word is associated with the specific lines it appears on. The indexer supports advanced search queries, including single-keyword searches and multi-keyword 'AND' operations. A key feature is the provision of detailed results, which include the lines containing the search terms and a summary of how many times each keyword appears, providing a comprehensive solution for text-based information retrieval.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the Text Search Indexer was approached in a sequential and modular manner. Initially, input texts were subjected to a rigorous tokenization and normalization process, converting text into a standardized format by lowercasing and removing non-alphanumeric characters to ensure consistent matching. Following normalization, a word-to-line index was constructed, mapping each unique word to a list of lines in which it appears. Search functionalities were then implemented, starting with single-keyword searches that leverage the index for direct lookup. Subsequently, a multi-keyword AND search was developed, requiring keywords to be present on the same line for a match. Finally, the system was enhanced to return search results augmented with occurrence count summaries for each keyword within the matched lines, providing a comprehensive view of search relevance.

6. Tools & Technologies Used

Programming Language: JavaScript

Libraries/Frameworks: Standard JavaScript features (String manipulation, Array methods, Object literals)

Concepts: Text processing, Tokenization, Normalization, Indexing, Data Structures (Hash Maps/Objects, Arrays), Search Algorithms, Information Retrieval.

7. Implementation & Results

Problem 1: Tokenize and Normalize Text

Description: The problem requires implementing a robust text processing pipeline that can handle diverse text inputs. This involves breaking down raw text into meaningful units (tokens) and transforming these tokens into a standardized format. Normalization typically includes converting text to lowercase and removing punctuation or special characters to ensure that variations in capitalization or spelling do not impede search accuracy. The goal is to prepare text data for effective indexing and searching by reducing its complexity and variability.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
  input = input.slice(0, -1);
}

const lines = input.split('\n');

const results = [];
for (const line of lines) {
  let result;
  if (line.trim() === '') {
    result = { "words": [] };
  } else {
    const words = line
      .toLowerCase()
      .replace(/^[a-z0-9\s]/g, "")
      .split(/\s+/)
      .filter(word => word.length > 0);

    result = { "words": words };
  }
  results.push(result);
}

console.log(JSON.stringify(results));
```

Problem 2: Build Word to Line Index

Description: This task focuses on creating an efficient data structure that maps words to their locations within the input texts. The objective is to build an index, specifically a word-to-line mapping, where each unique word found in the text is associated with a list of line numbers on which it occurs. This index serves as the backbone for fast retrieval of documents or lines containing specific terms, enabling quick lookups during search operations.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8').trim();
const lines = input.split('\n');

const index = {};

for (let i = 0; i < lines.length; i++) {
  const words = lines[i]
    .toLowerCase()
    .replace(/^[a-z0-9\s]/g, "")
    .split(/\s+/);

  for (const word of words) {
    if (word === "") continue;

    if (!index[word]) {
      index[word] = [];
    }

    // Avoid duplicate line numbers
    if (index[word].length === 0 || index[word][index[word].length - 1] !== i) {
      index[word].push(i);
    }
  }
}

console.log(JSON.stringify([index]));
```

Problem 3: Search for a Single Keyword

Description: The challenge is to develop a search functionality that can quickly locate all occurrences of a single user-specified keyword within the indexed text data. This involves querying the pre-built word-to-line index to retrieve all line numbers associated with the given keyword. The search must be efficient and accurate, returning the relevant lines where the keyword is present.

```
const fs = require('fs');

const keyword = fs.readFileSync(0, 'utf8').trim().toLowerCase();

const index = {
  hello: [0, 1],
  world: [0, 1]
};

console.log(JSON.stringify([index[keyword] || []]));
```

Problem 4: Support Multi-Keyword AND Search

Description: This problem extends the search capability to handle queries involving multiple keywords, specifically implementing an 'AND' logic. For a multi-keyword AND search, the system must identify lines that contain *all* specified keywords. This requires efficiently intersecting the sets of line numbers associated with each keyword in the index. The solution must ensure that only lines satisfying all keyword conditions are returned as results.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
  input = input.slice(0, -1);
}

const lines = input.split('\n');

const results = [];

const index = {"hello":[0,1], "world":[0,1], "test": [1]};
const keywords = lines[0].split(" ");

// --- YOUR COMPLETED SOLUTION LOGIC ---
let found_lines = [];

if (keywords.length > 0 && keywords[0] !== "") {
  // Initialize found_lines with the matches of the first keyword
  found_lines = index[keywords[0]] || [];

  // Intersect with the remaining keywords
  for (let i = 1; i < keywords.length; i++) {
    const currentKeyword = keywords[i];
    const currentLines = index[currentKeyword] || [];

    // Filter to keep only elements present in both arrays
    found_lines = found_lines.filter(line => currentLines.includes(line));
  }
}

// Push the intersected array into the results matrix
results.push(found_lines);

console.log(JSON.stringify(results));
```

Problem 5: Return Results with Occurrence Count Summary

Description: The final requirement is to enhance the search results by providing a summary of how many times each queried keyword appears on the lines that match the search criteria. Beyond simply listing the matching lines, the system should offer a quantitative measure of keyword frequency within the context of the search results, thereby adding significant value to the returned information.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
  input = input.slice(0, -1);
}

const lines = input.split('\n');

const results = [];

const index = {"hello":[0,1], "world":[0,1], "test": [1]};
const keywords = lines[0].split(" ");

// --- YOUR COMPLETED SOLUTION LOGIC ---
let found_lines = {};

for (let i = 0; i < keywords.length; i++) {
  const keyword = keywords[i];
  // Check if the keyword exists in the index
  if (index[keyword]) {
    // Count how many lines contain this keyword
    found_lines[keyword] = index[keyword].length;
  } else {
    // If keyword is missing from index, its line count is 0
    found_lines[keyword] = 0;
  }
}

results.push(found_lines);

console.log(JSON.stringify(results));
```

8. Learning Outcomes

- Gained practical experience in implementing text processing techniques such as tokenization and normalization.
- Developed a strong understanding of inverted indexing and its application in efficient text search.
- Learned to design and implement search algorithms for both single and multi-keyword queries.
- Enhanced skills in data structure manipulation, particularly using JavaScript objects and arrays for indexing.
- Improved problem-solving abilities in handling complex string operations and logical search conditions.

9. Challenges Faced & Solutions

Challenge: Ensuring case-insensitivity and handling of punctuation during tokenization and normalization.

Solution: Implemented a multi-step normalization process that converts text to lowercase and uses regular expressions to remove unwanted characters, ensuring consistent word matching.

Challenge: Efficiently performing multi-keyword AND searches across potentially large datasets.

Solution: Utilized set intersection operations on the lists of line numbers retrieved from the index for each keyword, allowing for rapid identification of lines containing all specified terms.

Challenge: Providing accurate occurrence counts for keywords within the context of search results.

Solution: After identifying matching lines, re-processed the text of those specific lines to count the occurrences of each keyword, rather than relying solely on the index which tracks presence rather than count.

10. Conclusion

The Text Search Indexer project successfully demonstrates a functional and efficient approach to text processing and information retrieval. By systematically addressing challenges in text normalization, indexing, and search, the system provides a robust tool for querying textual data. The implementation of single and multi-keyword searches, coupled with occurrence count summaries, highlights the practical application of core data structure and algorithm principles in solving real-world problems related to information access.

11. Future Scope

- * Implement OR and NOT search operators for more complex query capabilities.
- * Integrate fuzzy searching to handle misspellings and variations in keywords.
- * Extend the indexer to support searching across multiple documents or files concurrently.

12. References

- * JavaScript MDN Web Docs: String Methods - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String
- * JavaScript MDN Web Docs: Array Methods - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
- * GeeksforGeeks: Inverted Index - <https://www.geeksforgeeks.org/inverted-index/>

13. Annexure

- Internship Completion Certificate
- Offer Letter